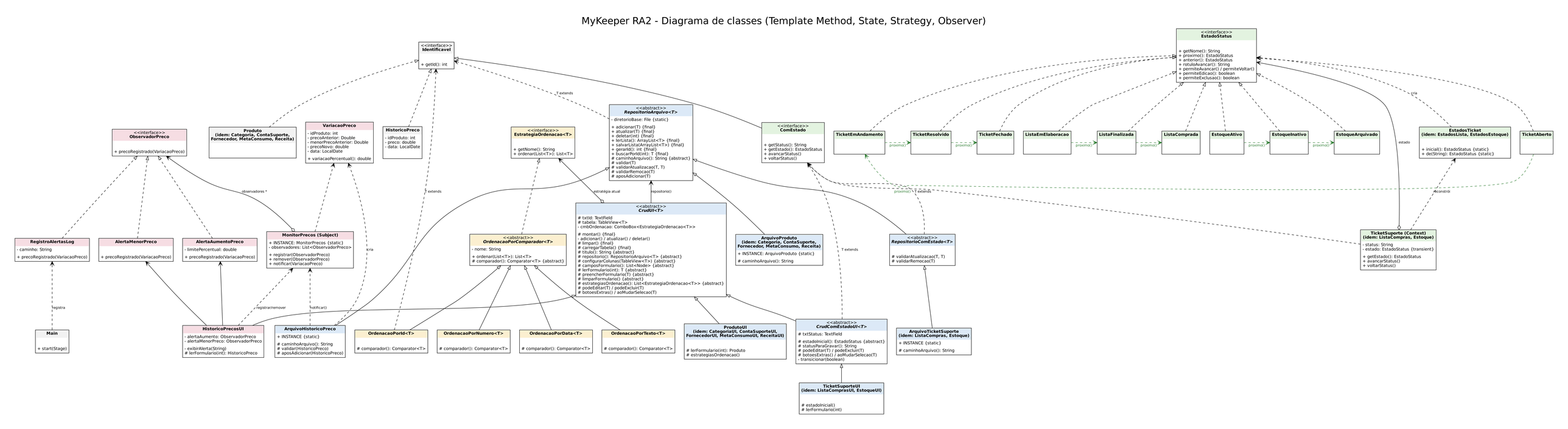
MyKeeper RA2 - Diagrama de classes (Template Method, State, Strategy, Observer)
<<interface>>
Identificavel
+ getId(): int
Produto
(idem: Categoria, ContaSuporte, Fornecedor, MetaConsumo, Receita)
VariacaoPreco
- idProduto: int
- precoAnterior: Double
- menorPrecoAnterior: Double
- precoNovo: double
- data: LocalDate
+ variacaoPercentual(): double
HistoricoPreco
- idProduto: int
- preco: double
- data: LocalDate
<<interface>>
ObservadorPreco
+ precoRegistrado(VariacaoPreco)
<<interface>>
EstrategiaOrdenacao<T>
+ getNome(): String
+ ordenar(List<T>): List<T>
<<abstract>>
RepositorioArquivo<T>
- diretorioBase: File {static}
+ adicionar(T) {final}
+ atualizar(T) {final}
+ deletar(int) {final}
+ lerLista(): ArrayList<T> {final}
+ salvarLista(ArrayList<T>) {final}
+ gerarId(): int {final}
+ buscarPorId(int): T {final}
# caminhoArquivo(): String {abstract}
# validar(T)
# validarAtualizacao(T, T)
# validarRemocao(T)
# aposAdicionar(T)
<<interface>>
ComEstado
+ getStatus(): String
+ getEstado(): EstadoStatus
+ avancarStatus()
+ voltarStatus()
<<interface>>
EstadoStatus
+ getNome(): String
+ proximo(): EstadoStatus
+ anterior(): EstadoStatus
+ rotuloAvancar(): String
+ permiteAvancar() / permiteVoltar()
+ permiteEdicao(): boolean
+ permiteExclusao(): boolean
TicketEmAndamento
TicketResolvido
TicketFechado
ListaEmElaboracao
ListaFinalizada
ListaComprada
EstoqueAtivo
EstoqueInativo
EstoqueArquivado
EstadosTicket
(idem: EstadosLista, EstadosEstoque)
+ inicial(): EstadoStatus {static}
+ delString): EstadoStatus {static}
TicketAberto
RegistroAlertasLog
- caminho: String
+ precoRegistrado(VariacaoPreco)
AlertaMenorPreco
+ precoRegistrado(VariacaoPreco)
AlertaAumentoPreco
- limitePercentual: double
+ precoRegistrado(VariacaoPreco)
MonitorPrecos (Subject)
+ INSTANCE: MonitorPrecos {static}
- observadores: List<ObservadorPreco>
+ registrar(ObservadorPreco)
+ remover(ObservadorPreco)
+ notificar(VariacaoPreco)
<<abstract>>
OrdenacaoPorComparador<T>
- nome: String
+ ordenar(List<T>): List<T>
# comparador(): Comparator<T> {abstract}
<<abstract>>
CrudUI<T>
txtId: TextField
tabela: TableView<T>
- cmbOrdenacao: ComboBox<EstrategiaOrdenacao<T>>
montar() {final}
- adicionar() / atualizar() / deletar()
limpar() {final}
carregarTabela() {final}
titulo(): String {abstract}
repositorio(): RepositorioArquivo<T> {abstract}
configurarColunas(TableView<T>) {abstract}
camposFormulario(): List<Node> {abstract}
lerFormulario(int): T {abstract}
preencherFormulario(T) {abstract}
limparFormulario() {abstract}
estrategiasOrdenacao(): List<EstrategiaOrdenacao<T>> {abstract}
podeEditar(T) / podeExcluir(T)
botoesExtras() / aoMudarSelecao(T)
ArquivoProduto
(idem: Categoria, ContaSuporte, Fornecedor, MetaConsumo, Receita)
+ INSTANCE: ArquivoProduto {static}
# caminhoArquivo(): String
<<abstract>>
RepositorioComEstado<T>
validarAtualizacao(T, T)
validarRemocao(T)
TicketSuporte (Context)
(idem: ListaCompras, Estoque)
- status: String
- estado: EstadoStatus {transient}
+ getEstado(): EstadoStatus
+ avancarStatus()
+ voltarStatus()
Main
+ start(Stage)
HistoricoPrecosUI
- alertaAumento: ObservadorPreco
- alertaMenorPreco: ObservadorPreco
- exibirAlerta(String)
lerFormulario(int): HistoricoPreco
ArquivoHistoricoPreco
+ INSTANCE {static}
# caminhoArquivo(): String
# validar(HistoricoPreco)
# aposAdicionar(HistoricoPreco)
OrdenacaoPorId<T>
comparador(): Comparator<T>
OrdenacaoPorNumero<T>
comparador(): Comparator<T>
OrdenacaoPorData<T>
comparador(): Comparator<T>
OrdenacaoPorTexto<T>
comparador(): Comparator<T>
ProdutoUI
(idem: CategoriaUI, ContaSuporteUI, FornecedorUI, MetaConsumoUI, ReceitaUI)
lerFormulario(int): Produto
estrategiasOrdenacao()
<<abstract>>
CrudComEstadoUI<T>
txtStatus: TextField
estadoInicial(): EstadoStatus {abstract}
statusParaGravar(): String
podeEditar(T) / podeExcluir(T)
botoesExtras() / aoMudarSelecao(T)
- transicionar(boolean)
ArquivoTicketSuporte
(idem: ListaCompras, Estoque)
+ INSTANCE {static}
# caminhoArquivo(): String
TicketSuporteUI
(idem: ListaComprasUI, EstoqueUI)
estadoInicial()
lerFormulario(int)
observadores *
T extends
estratégia atual
repositorio()
registra
registrar/remover
notificar()
cria
estado
reconstrói
proximo()